

# 【2026年最新版】 Gemini CLIで行うべきセキュリティ設定 10選

🔗 来源: <https://qiita.com/miruky/items/3f091416cfbfbff99eb>

Transform your terminal  
today with Gemini CLI

こんばんは、mirukyです。

別記事で [Claude Codeのセキュリティ設定についてまとめました](#) が、自分がかつて Gemini CLIも使用してみたことがあるので、これを機に再度触ってまとめてみることにしました。

皆さんご存知の通り、Gemini CLIはGoogleが提供するオープンソースのAIコーディングエージェントですが、シェルコマンドの実行やファイルの読み書きができるため、セキュリティ設定を怠ると思わぬリスクを招きます。

本記事では、Gemini CLIを安全に使うために **今すぐ設定すべき10項目** を、公式ドキュメントに基づいて紹介します。

本記事は2026年3月時点のGemini CLI公式ドキュメント (GitHub: [google-gemini/gemini-cli](#)) に基づいています。

## ① サンドボックスを有効化する

最も重要な設定です。Gemini CLIはデフォルトでサンドボックスが **無効** になっています。有効化すると、ツール実行がOS レベルで隔離され、ファイルシステムやネットワークへのアクセスが制限されます。

有効化には3つの方法があります：

```
# 方法1: 起動時フラグ (最も優先度が高い)
```

```
gemini --sandbox
```

```
gemini -s
```

```
# 方法2: 環境変数
```

```
export GEMINI_SANDBOX=true
```

```
// 方法3: ~/.gemini/settings.json (永続的)
```

```
{
  "tools": {
    "sandbox": "docker"
  }
}
```

サンドボックスの種類は以下の4つです：

| 方式                             | OS    | 特徴                          |
|--------------------------------|-------|-----------------------------|
| <b>Seatbelt</b> (sandbox-exec) | macOS | デフォルト。プロファイルで細かく制御可能        |
| <b>Docker / Podman</b>         | 全OS   | コンテナベース。カスタムDockerfileも利用可能 |
| <b>gVisor</b> (runsc)          | Linux | 最も強力な隔離。カーネルレベルで仮想化         |
| <b>LXC / LXD</b>               | Linux | 実験的。軽量コンテナ                  |

**--yolo** モード使用時はサンドボックスが **自動有効化** されますが、通常使用時はデフォルト無効です。まだ有効化していない方は、最優先で設定してください。

## ② macOSではSeatbeltプロファイルを強化する

macOSのデフォルトプロファイルは **permissive-open** (最も緩い設定) です。セキュリティを重視するなら、**restrictive**以上のプロファイルに切り替えましょう。

```
# restrictive-open: デフォルトで操作を拒否、ネットワークは許可
export SEATBELT_PROFILE=restrictive-open

# strict-open: 読み書きともワーキングディレクトリに制限、ネットワークは許可
export SEATBELT_PROFILE=strict-open

# strict-proxied: 最も厳しい設定。ネットワークもプロキシ経由に制限
export SEATBELT_PROFILE=strict-proxied
```

6つのプロファイルの違い:

| プロファイル              | 書き込み制限        | 読み取り制限        | ネットワーク |
|---------------------|---------------|---------------|--------|
| permissive-open     | プロジェクトフォルダのみ  | 制限なし          | 直接     |
| permissive-proxied  | プロジェクトフォルダのみ  | 制限なし          | プロキシ経由 |
| restrictive-open    | デフォルト拒否       | デフォルト拒否       | 直接     |
| restrictive-proxied | デフォルト拒否       | デフォルト拒否       | プロキシ経由 |
| strict-open         | ワーキングディレクトリのみ | ワーキングディレクトリのみ | 直接     |
| strict-proxied      | ワーキングディレクトリのみ | ワーキングディレクトリのみ | プロキシ経由 |

カスタムプロファイルを作成する場合は、`.gemini/sandbox-macos-<プロファイル名>.sb`に配置します。

### ③ YOLOモードを無効化する

`--yolo` フラグ ( `--approval-mode=yolo` ) は、すべてのツール実行を自動承認するモードです。チーム開発では特に、このモードを 完全に使用不可 にしておくべきです。

```
// ~/.gemini/settings.json または システム設定ファイル
{
  "security": {
    "disableYoloMode": true
  }
}
```

この設定を システム設定ファイル（後述の⑩）に配置すると、ユーザー側で上書きできません。

代わりに `--approval-mode=auto_edit`（編集ツールのみ自動承認）や `plan`（読み取り専用モード）を活用しましょう。

## ④ 「Always Allow」オプションを無効化する

ツール実行の確認ダイアログで「Always allow」を選択すると、以降そのツールは確認なしで実行されます。これが蓄積すると、意図しない操作が自動承認されるリスクが高まります。

```
{
  "security": {
    "disableAlwaysAllow": true
  }
}
```

さらに厳格にするなら、Adminセキュアモードで一括制御できます：

```
{
  "admin": {
    "secureModeEnabled": true
  }
}
```

`admin.secureModeEnabled: true` は、YOLOモードと「Always allow」の両方を一括で無効化します。

## ⑤ 実行可能なツールをホワイトリストで制限する

`tools.core` 設定で、使用可能なツールを ホワイトリスト方式 で制限できます。ブロックリスト方式（`tools.exclude`）よりも安全です。

```
{
  "tools": {
    "core": [
      "ReadFileTool",
      "GlobTool",
      "ShellTool(ls)",
      "ShellTool(cat)",
      "ShellTool(grep)",
      "ShellTool(git status)",
      "ShellTool(git diff)"
    ]
  }
}
```

また、`tools.allowed` で特定のコマンドのみ確認ダイアログをスキップできます:

```
{
  "tools": {
    "allowed": [
      "run_shell_command(git status)",
      "run_shell_command(npm test)"
    ]
  }
}
```

ホワイトリスト（`tools.core`）が推奨です。ブロックリスト（`tools.exclude`）は既知の危険コマンドしかブロックできず、巧妙な回避が可能です。

## ⑥ 環境変数のリダクション（秘匿化）を有効化する

Gemini CLIには、シェルコマンド実行時に環境変数に含まれるシークレットを自動的にリダクション（マスク）する機能があります。

```
{
  "security": {
    "environmentVariableRedaction": {
      "enabled": true,
      "blocked": [
        "DATABASE_URL",
        "INTERNAL_API_KEY"
      ],
    },
  },
}
```

```
    "allowed": [
      "MY_PUBLIC_CONFIG"
    ]
  }
}
```

デフォルトのリダクションルール:

- 変数名ベース: `TOKEN`, `SECRET`, `PASSWORD`, `KEY`, `AUTH`, `CREDENTIAL` 等を含む変数名を検出
- 値ベース: RSA秘密鍵、GitHubトークン、AWSキー、Stripeキー等のパターンマッチ
- 固定ブロックリスト: `CLIENT_ID`, `DB_URI`, `DATABASE_URL`, `CONNECTION_STRING` は常に秘匿

`PATH`, `HOME`, `USER`, `SHELL` などのシステム変数や `GEMINI_CLI_` で始まる変数は自動的に除外されます。

公式ドキュメントでは代替キー名として `security.allowedEnvironmentVariables` (許可リスト) と `security.blockedEnvironmentVariables` (ブロックリスト) も定義されています。どちらの記法でも設定可能です。

## ⑦ Trusted Foldersでフォルダ信頼性を管理する

Gemini CLIでは、プロジェクトフォルダの信頼レベルを管理できます。この機能はデフォルトで有効です (`security.folderTrust.enabled: true`)。信頼されていないフォルダでは以下が自動的に無効化されます:

- ワークスペース設定ファイルの読み込み
- 環境変数の読み込み
- ツールの自動承認
- メモリの自動読み込み
- MCPサーバーへの接続
- カスタムコマンドの読み込み
- 拡張機能の管理

信頼設定は `~/.gemini/trustedFolders.json` に保存されます。セッション中に `/permissions` コマンドで信頼レベルを変更できます。もし無効化されている場合は、以下で再有効化できます:

```
// ~/.gemini/settings.json
{
  "security": {
    "folderTrust": {
      "enabled": true
    }
  }
}
```

出所不明のリポジトリをクローンした場合、初回起動時にフォルダの信頼確認ダイアログが表示されます。安易に「**Trust**」を選択しないことが重要です。

## ⑧ BeforeToolフックでカスタム安全チェックを追加する

Hooksを使えば、ツール実行の前後にカスタムスクリプトを実行できます。

`BeforeTool` フックを使えば、ツール実行前に独自のバリデーションを挟めます。

```
// ~/.gemini/settings.json
{
  "hooks": {
    "BeforeTool": [
      {
        "command": ".gemini/hooks/validate-tool.sh",
        "description": "Validate tool execution for security"
      }
    ]
  }
}
```

フックスクリプトの例（危険なコマンドをブロック）:

```
#!/bin/bash
# .gemini/hooks/validate-tool.sh

# stdinからツール情報を読み取り
TOOL_INPUT=$(cat /dev/stdin)
COMMAND=$(echo "$TOOL_INPUT" | jq -r '.command // empty')
```

```

# rm -rf をブロック
if echo "$COMMAND" | grep -q 'rm -rf'; then
    echo "Blocked: rm -rf commands are not allowed" >&2
    exit 1
fi

# 本番環境への接続をブロック
if echo "$COMMAND" | grep -q 'production'; then
    echo "Blocked: production access is not allowed" >&2
    exit 1
fi

exit 0

```

Gemini CLIのフックは以下のライフサイクルポイントで利用可能です:

| フック                                                 | タイミング           |
|-----------------------------------------------------|-----------------|
| <code>BeforeTool</code> / <code>AfterTool</code>    | ツール実行の前後        |
| <code>BeforeAgent</code> / <code>AfterAgent</code>  | エージェントループの開始・終了 |
| <code>BeforeModel</code> / <code>AfterModel</code>  | LLMリクエストの前後     |
| <code>SessionStart</code> / <code>SessionEnd</code> | セッションの開始・終了     |
| <code>PreCompress</code>                            | チャット履歴圧縮の前      |
| <code>Notification</code>                           | 通知イベント発生時       |
| <code>BeforeToolSelection</code>                    | ツール選択の前         |

## ⑨ MCPサーバーをホワイトリストで管理する

MCPサーバーは外部ツールを統合する強力な機能ですが、信頼できないサーバーは深刻なセキュリティリスクになります。`mcp.allowed`でホワイトリストを設定しましょう。

```

{
  "mcp": {
    "allowed": ["corp-tools", "code-analyzer"]
  },
  "mcpServers": {
    "corp-tools": {
      "command": "/usr/local/bin/corp-tools.sh",
      "timeout": 5000,

```



```

    "includeTools": [ "code-search", "get-ticket-details" ]
  },
  "code-analyzer": {
    "command": "node",
    "args": [ "analyzer-server.js" ],
    "excludeTools": [ "delete-file", "execute-arbitrary" ]
  }
}
}

```

ポイント:

- `mcp.allowed` を必ず設定: 省略すると、ユーザーが自由にサーバーを追加可能
- `includeTools` でツール単位ホワイトリストを使う (最小権限の原則)
- `excludeTools` は `includeTools` より優先: 両方に含まれるツールは除外される
- サーバーエイリアスに アンダースコア ( `_` ) を使わない: ポリシーエンジンの FQN 解析が誤動作する

管理者は `admin.mcp.enabled: false` で、MCPサーバーの使用自体を完全に禁止できます。

## ⑩ Enterprise: システム設定ファイルで組織ポリシーを強制する

チームや組織で Gemini CLI を使う場合は、システム設定ファイルで組織ポリシーを一括管理できます。システム設定ファイルは **最高の優先度** を持ち、ユーザーやプロジェクトの設定で上書きできません。

設定ファイルの優先度:

| 優先度   | ファイル                    | 説明                                   |
|-------|-------------------------|--------------------------------------|
| 1 (低) | System Defaults         | システム全体のデフォルト値                        |
| 2     | User Settings           | <code>~/.gemini/settings.json</code> |
| 3     | Project Settings        | <code>.gemini/settings.json</code>   |
| 4 (高) | <b>System Overrides</b> | 管理者が強制する設定                           |

システム設定ファイルのパス:

| OS      | SYSTEM OVERRIDES のパス                                 |
|---------|------------------------------------------------------|
| macOS   | /Library/Application Support/GeminiCli/settings.json |
| Linux   | /etc/gemini-cli/settings.json                        |
| Windows | C:\ProgramData\gemini-cli\settings.json              |

組織管理者向けの推奨設定例:

```
{
  "tools": {
    "sandbox": "docker",
    "core": [
      "ReadFileTool",
      "GlobTool",
      "ShellTool(ls)",
      "ShellTool(cat)",
      "ShellTool(grep)"
    ]
  },
  "security": {
    "disableYoloMode": true,
    "disableAlwaysAllow": true,
    "environmentVariableRedaction": {
      "enabled": true
    }
  },
  "mcp": {
    "allowed": ["corp-tools"]
  },
  "mcpServers": {
    "corp-tools": {
      "command": "/opt/gemini-tools/start.sh",
      "timeout": 5000
    }
  },
  "telemetry": {
    "enabled": true,
    "target": "gcp",
    "logPrompts": false
  },
  "privacy": {
    "usageStatisticsEnabled": false
  }
}
```

```
}
```

さらに、ラッパースクリプトで環境変数を強制することもできます:

```
#!/bin/bash
# /usr/local/bin/gemini (ラッパースクリプト)
export GEMINI_CLI_SYSTEM_SETTINGS_PATH="/etc/gemini-
cli/settings.json"

REAL_GEMINI_PATH=$(type -aP gemini | grep -v "^$(type -P gemini)$"
| head -n 1)
exec "$REAL_GEMINI_PATH" "$@"
```

## まとめ

適切なセキュリティ設定をして、思う存分Gemini CLIを満喫しましょう。  
ではまた、お会いしましょう。

## 参考リンク

1

[コメント一覧へ移動](#)

[X \(Twitter\) でシェアする](#)

[Facebookでシェアする](#)

[はてなブックマークに追加する](#)